

在 Cloud AI 平台訓練 PyTorch 模型及調整超參數

來源：<https://codelabs.developers.google.com/codelabs/training-tuning-caip?hl=zh-tw>

步驟數：7

本研究室將說明如何使用超參數調整功能在 Cloud 中訓練模型。我們將說明如何使用 PyTorch 執行這項作業，但您可以自行選擇任何架構中的做法。

目錄

1. [1. 總覽](#)
2. [2. 設定環境](#)
3. [3. 建立訓練工作的套件](#)
4. [4. 預覽資料集](#)
5. [5. 定義採用超參數調整的訓練工作](#)
6. [6. 在 AI Platform 上執行訓練工作](#)
7. [7. 清除所用資源](#)

步驟 1 · 約 1 分鐘

1. 總覽

在本實驗室中，您將逐步瞭解 Google Cloud 上的完整機器學習訓練工作流程，並使用 PyTorch 建構模型。您將在 Cloud AI Platform Notebooks 環境中，瞭解如何封裝訓練工作，以便在 AI Platform Training 上運作執行，並進行超參數調整。

課程內容

內容如下：

- 建立 AI 平台 Notebooks 執行個體
- 建立 PyTorch 模型
- 在 AI Platform Training 上訓練模型並調整超參數

在 Google Cloud 上執行這個實驗室的總費用約為 **\$1 美元**。

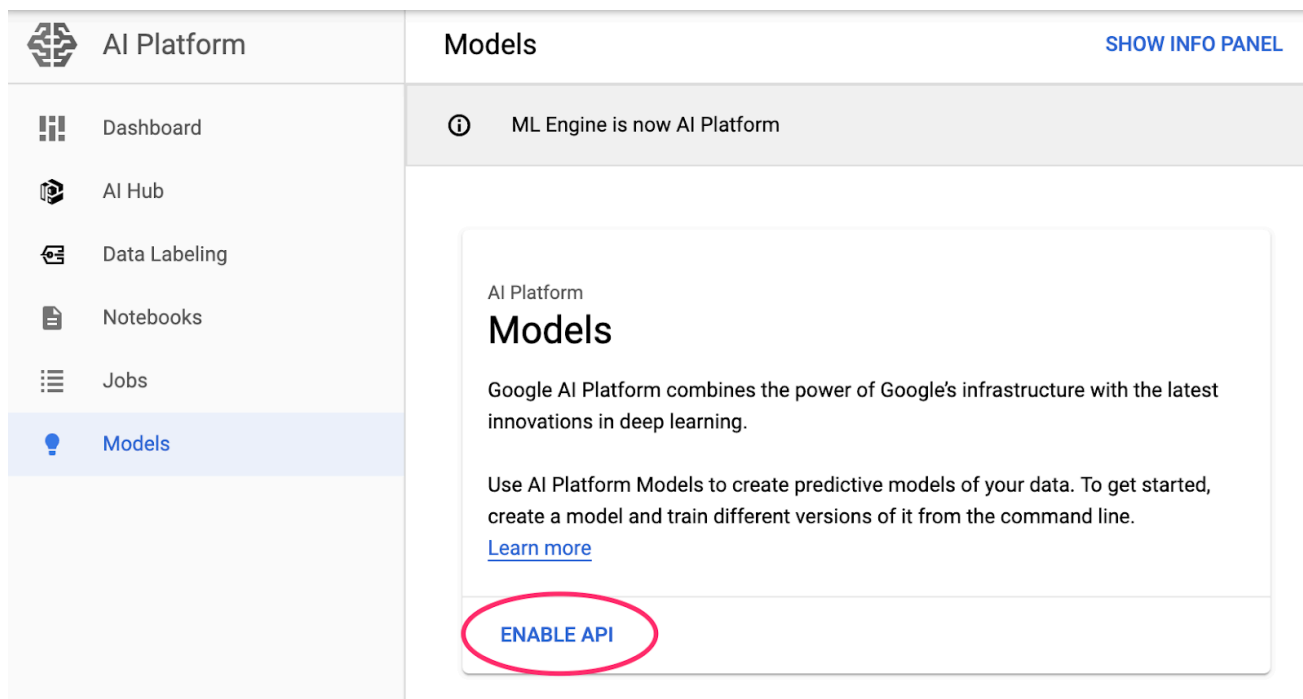
步驟 2 · 約 10 分鐘

2. 設定環境

您必須擁有已啟用計費功能的 Google Cloud Platform 專案，才能執行這項程式碼研究室。如要建立專案，請按照[這裡](#)的操作說明進行。

步驟 1：啟用 Cloud AI Platform Models API

前往 Cloud 控制台的 [AI Platform Models 區段](#)，如果尚未啟用，請點選「啟用」。

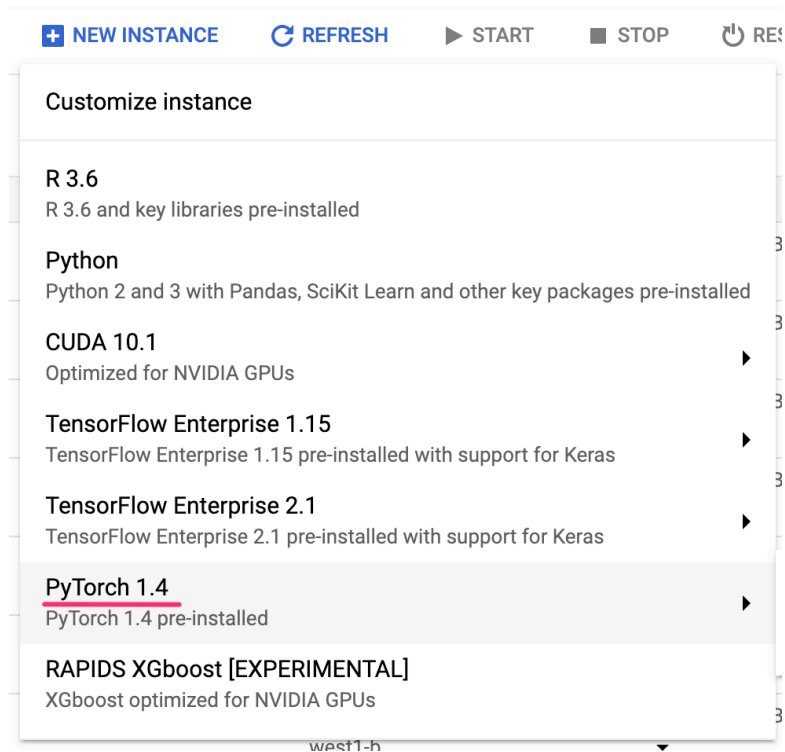


步驟 2：啟用 Compute Engine API

前往「Compute Engine」，然後選取「啟用」（如果尚未啟用）。您需要這項資訊才能建立筆記本執行個體。

步驟 3：建立 AI Platform Notebooks 執行個體

前往 Cloud Console 的 [AI Platform Notebooks 專區](#)，然後點選「建立執行個體」。然後選取最新 **PyTorch** 執行個體類型 (不含 GPU)：

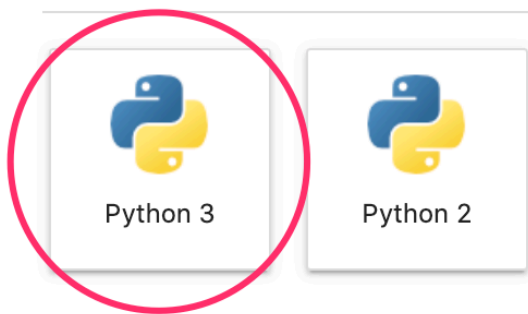


使用預設選項，或視需要輸入自訂名稱，然後按一下「建立」。建立執行個體後，請選取「Open JupyterLab」：

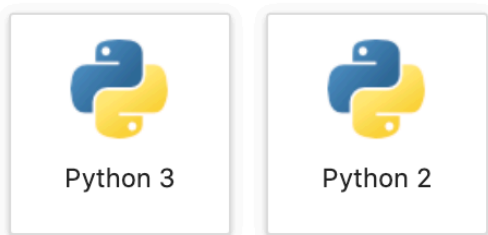


接著，從啟動器開啟 Python 3 筆記本執行個體：

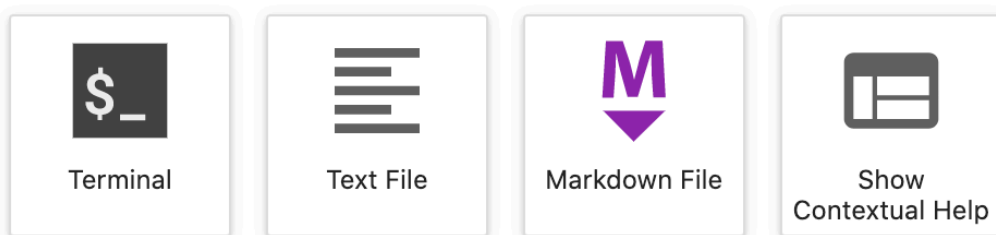
Notebook



Console



Other



你已準備就緒，可以開始使用了！

步驟 5：匯入 Python 套件

在本程式碼研究室的其餘部分，請從 Jupyter 筆記本執行所有程式碼片段。

在筆記本的第一個儲存格中，新增下列匯入內容並執行儲存格。如要執行，請按下頂端選單中的向右箭頭按鈕，或按下 Command-Enter 鍵：

```
import datetime
import numpy as np
import os
import pandas as pd
import time
```

你會發現我們在這裡並未匯入 PyTorch。這是因為我們是在 AI Platform Training 上執行訓練工作，而不是從 Notebook 執行個體執行。

步驟 3 · 約 5 分鐘

3. 建立訓練工作的套件

如要在 AI Platform Training 上執行訓練工作，我們需要將訓練程式碼封裝在 Notebooks 執行個體本機，並建立 Cloud Storage 值區來儲存工作資產。首先，我們將建立儲存空間 bucket。如果已有帳戶，可以略過這個步驟。

步驟 1：為模型建立 Cloud Storage bucket

首先，請定義一些環境變數，我們會在程式碼研究室的其餘部分使用這些變數。請在下方填入 Google Cloud 雲端專案名稱，以及您要建立的 Cloud Storage bucket 名稱 (必須是全域不重複的名稱)：

```
# Update these to your own GCP project, model, and version names
GCP_PROJECT = 'your-gcp-project'
BUCKET_URL = 'gs://storage_bucket_name'
```

現在可以建立儲存空間值區，啟動訓練工作時會指向這個值區。

在筆記本中執行下列 `gsutil` 指令，建立 bucket：

```
!gsutil mb $BUCKET_URL
```

步驟 2：為 Python 套件建立初始檔案

如要在 AI Platform 上執行訓練工作，我們需要將程式碼設定為 Python 套件。這包括根目錄中的 `setup.py` 檔案 (指定任何外部套件依附元件)、以套件名稱命名的子目錄 (在此我們將其命名為 `trainer/`)，以及這個子目錄中的空白 `__init__.py` 檔案。

首先，請編寫 `setup.py` 檔案。我們使用 `iPython %%writefile` magics 將檔案儲存至執行個體。我們在此指定了訓練程式碼中會使用的 3 個外部程式庫：PyTorch、Scikit-learn 和 Pandas：

```
%%writefile setup.py
from setuptools import find_packages
from setuptools import setup

REQUIRED_PACKAGES = ['torch>=1.5', 'scikit-learn>=0.20', 'pandas>=1.0']

setup(
    name='trainer',
    version='0.1',
    install_requires=REQUIRED_PACKAGES,
    packages=find_packages(),
    include_package_data=True,
    description='My training application package.'
)
```

接著，我們要在其中建立 `trainer/` 目錄和空白的 `init.py` 檔案。Python 會使用這個檔案，確認這是套件：

```
!mkdir trainer
!touch trainer/__init__.py
```


現在可以開始建立訓練作業了。

4. 預覽資料集

本實驗室的重點是訓練模型的工具，但我們先快速瞭解用於訓練模型的資料集。我們將使用 BigQuery 中提供的[生育率資料集](#)。其中包含美國數十年的出生資料。我們將使用資料集中的幾個資料欄，預測嬰兒出生體重。原始資料集相當龐大，我們會使用其中的子集，並將其放在 Cloud Storage bucket 中供您使用。

步驟 1：下載 BigQuery 出生率資料集

讓我們將 Cloud Storage 中提供的資料集版本下載至 Pandas DataFrame，並預覽資料集。

```
natality = pd.read_csv('https://storage.googleapis.com/ml-design-patterns/natality.csv')
natality.head()
```

這個資料集有將近 10 萬列。我們會使用 5 項特徵來預測嬰兒的出生體重：母親和父親的年齡、妊娠週數、母親體重增加的磅數，以及以布林值表示的嬰兒性別。

步驟 5 · 約 10 分鐘

5. 定義採用超參數調整的訓練工作

我們會在稍早建立的 `trainer/` 子目錄中，將訓練指令碼寫入名為 `model.py` 的檔案。訓練工作會在 AI Platform Training 上執行，並使用 AI Platform 的超參數調整服務，透過貝氏最佳化方法找出模型的最佳超參數。

步驟 1：建立訓練指令碼

首先，請使用訓練指令碼建立 Python 檔案。然後剖析其中的內容。執行這個 `%%writefile` 指令，即可將模型程式碼寫入本機 Python 檔案：

```

%%writefile trainer/model.py
import argparse
import hypertune
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import torch.optim as optim

from sklearn.utils import shuffle
from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import normalize

def get_args():
    """Argument parser.
    Returns:
        Dictionary of arguments.
    """
    parser = argparse.ArgumentParser(description='PyTorch MNIST')
    parser.add_argument('--job-dir', # handled automatically by AI Platform
                        help='GCS location to write checkpoints and export ' \
                             'models')
    parser.add_argument('--lr', # Specified in the config file
                        type=float,
                        default=0.01,
                        help='learning rate (default: 0.01)')
    parser.add_argument('--momentum', # Specified in the config file
                        type=float,
                        default=0.5,
                        help='SGD momentum (default: 0.5)')
    parser.add_argument('--hidden-layer-size', # Specified in the config file
                        type=int,
                        default=8,
                        help='hidden layer size')

    args = parser.parse_args()
    return args

def train_model(args):
    # Get the data
    natality = pd.read_csv('https://storage.googleapis.com/ml-design-patterns/natality.csv')
    natality = natality.dropna()
    natality = shuffle(natality, random_state = 2)
    natality.head()

    natality_labels = natality['weight_pounds']
    natality = natality.drop(columns=['weight_pounds'])

    train_size = int(len(natality) * 0.8)
    traindata_natality = natality[:train_size]
    trainlabels_natality = natality_labels[:train_size]

```

```

testdata_natality = natality[train_size:]
testlabels_natality = natality_labels[train_size:]

# Normalize and convert to PT tensors
normalized_train = normalize(np.array(traindata_natality.values), axis=0)
normalized_test = normalize(np.array(testdata_natality.values), axis=0)

train_x = torch.Tensor(normalized_train)
train_y = torch.Tensor(np.array(trainlabels_natality))

test_x = torch.Tensor(normalized_test)
test_y = torch.Tensor(np.array(testlabels_natality))

# Define our data loaders
train_dataset = torch.utils.data.TensorDataset(train_x, train_y)
train_dataloader = torch.utils.data.DataLoader(train_dataset, batch_size=128,
shuffle=True)

test_dataset = torch.utils.data.TensorDataset(test_x, test_y)
test_dataloader = torch.utils.data.DataLoader(test_dataset, batch_size=128,
shuffle=False)

# Define the model, while tuning the size of our hidden layer
model = nn.Sequential(nn.Linear(len(train_x[0]), args.hidden_layer_size),
                      nn.ReLU(),
                      nn.Linear(args.hidden_layer_size, 1))
criterion = nn.MSELoss()

# Tune hyperparameters in our optimizer
optimizer = optim.SGD(model.parameters(), lr=args.lr, momentum=args.momentum)
epochs = 20
for e in range(epochs):
    for batch_id, (data, label) in enumerate(train_dataloader):
        optimizer.zero_grad()
        y_pred = model(data)
        label = label.view(-1,1)
        loss = criterion(y_pred, label)

        loss.backward()
        optimizer.step()

val_mse = 0
num_batches = 0
# Evaluate accuracy on our test set
with torch.no_grad():
    for i, (data, label) in enumerate(test_dataloader):
        num_batches += 1
        y_pred = model(data)
        mse = criterion(y_pred, label.view(-1,1))
        val_mse += mse.item()

```

```

avg_val_mse = (val_mse / num_batches)

# Report the metric we're optimizing for to AI Platform's HyperTune service
# In this example, we're minimizing error on our test set
hpt = hypertune.HyperTune()
hpt.report_hyperparameter_tuning_metric(
    hyperparameter_metric_tag='val_mse',
    metric_value=avg_val_mse,
    global_step=epochs
)

def main():
    args = get_args()
    print('in main', args)
    train_model(args)

if __name__ == '__main__':
    main()

```

訓練工作包含兩個函式，大部分的工作都在這兩個函式中進行。

- `get_args()`：這會剖析我們建立訓練工作時傳遞的指令列引數，以及我們希望 AI Platform 最佳化的超參數。在這個範例中，引數清單只包含我們要最佳化的超參數，也就是模型的學習率、動量和隱藏層中的神經元數量。
- `train_model()`：我們在此將資料下載至 Pandas DataFrame、進行正規化、轉換為 PyTorch 張量，然後定義模型。我們使用 PyTorch `nn.Sequential` API 建構模型，可將模型定義為一疊層：

```

model = nn.Sequential(nn.Linear(len(train_x[0]), args.hidden_layer_size),
                      nn.ReLU(),
                      nn.Linear(args.hidden_layer_size, 1))

```

請注意，我們並未對模型隱藏層的大小進行硬式編碼，而是將其設為超參數，由 AI Platform 為我們調整。詳情請見下一節。

步驟 2：使用 AI Platform 的超參數調整服務

我們不會手動嘗試不同的超參數值，然後每次都重新訓練模型，而是使用 Cloud AI Platform 的[超參數最佳化服務](#)。如果我們使用超參數引數設定訓練工作，AI Platform 會使用[貝氏最佳化](#)，找出指定超參數的理想值。

在超參數調整中，單一「試驗」是指使用特定超參數值組合訓練模型一次。視執行的試驗次數而定，AI 平台會使用已完成試驗的結果，為日後的試驗選取最佳超參數。如要設定超參數調整，我們必須在啟動訓練工作時傳遞設定檔，其中包含要最佳化的每個超參數資料。

接著，在本機建立該設定檔：

```
%%writefile config.yaml
trainingInput:
  hyperparameters:
    goal: MINIMIZE
    maxTrials: 10
    maxParallelTrials: 5
    hyperparameterMetricTag: val_mse
    enableTrialEarlyStopping: TRUE
    params:
      - parameterName: lr
        type: DOUBLE
        minValue: 0.0001
        maxValue: 0.1
        scaleType: UNIT_LINEAR_SCALE
      - parameterName: momentum
        type: DOUBLE
        minValue: 0.0
        maxValue: 1.0
        scaleType: UNIT_LINEAR_SCALE
      - parameterName: hidden-layer-size
        type: INTEGER
        minValue: 8
        maxValue: 32
        scaleType: UNIT_LINEAR_SCALE
```

針對每個超參數，我們指定了類型、要搜尋的值範圍，以及在不同實驗中增加值的比例。

在工作開始時，我們也會指定要進行最佳化的指標。請注意，在上述 `train_model()` 函式結尾，我們會在每次試驗完成時，向 AI 平台 回報這項指標。我們在這裡要盡量減少模型的均方誤差，因此要使用可讓模型產生最低均方誤差的超參數。這項指標的名稱 (`val_mse`) 與我們在試用期結束時呼叫 `report_hyperparameter_tuning_metric()` 時使用的名稱相同。

注意：我們不會直接在筆記本執行模型訓練作業，但您可以將模型訓練程式碼複製到新儲存格，並在筆記本中試用。只要將所有 `args` 參照替換為硬式編碼的超參數值即可。您已建立 PyTorch 執行個體，現在可以開始使用。

步驟 6 · 約 20 分鐘

6. 在 AI Platform 上執行訓練工作

在本節中，我們將在 AI 平台 上啟動超參數調整，開始模型訓練工作。

步驟 1：定義一些環境變數

首先，請定義一些環境變數，用於啟動訓練工作。如要在其他區域執行工作，請更新下方的 REGION 變數：

```
MAIN_TRAINER_MODULE = "trainer.model"
TRAIN_DIR = os.getcwd() + '/trainer'
JOB_DIR = BUCKET_URL + '/output'
REGION = "us-central1"
```

AI Platform 上的每項訓練工作都應有不重複的名稱。執行下列指令，使用時間戳記定義工作名稱的變數：

```
timestamp = str(datetime.datetime.now().time())
JOB_NAME = 'caip_training_' + str(int(time.time()))
```

步驟 2：啟動訓練工作

我們會使用 Google Cloud CLI (gcloud) 建立訓練作業。我們可以直接在筆記本中執行這項指令，並參照上方定義的變數：

```
!gcloud ai-platform jobs submit training $JOB_NAME \
  --scale-tier basic \
  --package-path $TRAIN_DIR \
  --module-name $MAIN_TRAINER_MODULE \
  --job-dir $JOB_DIR \
  --region $REGION \
  --runtime-version 2.1 \
  --python-version 3.7 \
  --config config.yaml
```

如果工作建立正確無誤，請前往 AI Platform 控制台的「Jobs」(工作) [區段](#) 監控記錄。

步驟 3：監控工作

進入控制台的「Jobs」部分後，按一下剛開始的工作即可查看詳細資料：

Jobs + NEW TRAINING JOB BETA REFRESH CANCEL								
Filter by prefix...								
☐	Job ID	Type	HyperTune	HyperTune parameters	Target metric	Create time	Elapsed time	Logs
☐	 caip_training_1590072136	Custom code training	Yes	lr, momentum, hidden-layer-size	val_mse	May 21, 2020, 10:42:24 AM	13 sec	View Logs

第一輪試驗開始後，您就能看到為每個試驗選取的超參數值：

HyperTune trials

	Trial ID	val_mse ↑	Training step	lr	momentum	hidden-layer-size	
○	5			0.03848	0.46064	19	⋮
○	4			0.0418	0.0142	25	⋮
○	3			0.0978	0.25852	13	⋮
○	2			0.05614	0.54639	16	⋮
○	1			0.08944	0.94639	28	⋮

試用期結束後，系統會在此記錄最佳化指標的結果值 (在本例中為 `val_mse`)。這項工作應會執行 15 到 20 分鐘，完成後資訊主頁會如下所示 (實際值會有所不同)：

HyperTune trials

	Trial ID	val_mse ↑	Training step	lr	momentum	hidden-layer-size	
○	8	1.57975	10	0.03077	0.56037	28	⋮
○	1	1.58012	10	0.08775	0.45065	21	⋮
○	10	1.58028	10	0.01827	0.71427	21	⋮
○	5	1.58127	10	0.00938	0.87626	23	⋮
○	3	1.58167	10	0.02115	0.85065	27	⋮
○	6	1.58437	10	0.0054	0.98399	19	⋮
○	4	1.58805	10	0.08675	0.66243	21	⋮
○	7	1.61209	10	0.06407	0.96037	16	⋮
○	9	1.65974	10	0.05157	0.11427	9	⋮
○	2	2.11363	10	0.05445	0.05065	9	⋮

如要偵錯潛在問題並更詳細地監控工作，請在工作詳細資料頁面中按一下「查看記錄檔」：

caip_training_1590072136

Running (2 min 16 sec)

Creation time May 21, 2020, 10:42:24 AM

Start time May 21, 2020, 10:42:29 AM

End time

Logs [View Logs](#)

Training input [SHOW JSON](#)

Training output [SHOW JSON](#)

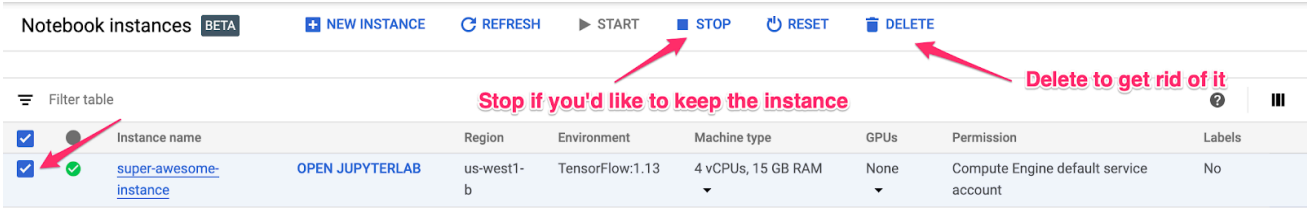
模型訓練程式碼中的每個 `print()` 陳述式都會顯示在這裡。如果遇到問題，請嘗試新增更多列印陳述式，然後啟動新的訓練工作。

訓練工作完成後，找出產生最低 `val_mse` 的超參數。您可以運用這些結果訓練及匯出模型的最終版本，也可以根據這些結果啟動其他訓練工作，並進行額外的超參數調整試驗。

想使用其他架構建構模型嗎？我們在此說明如何訓練 PyTorch 模型，但使用 *任何其他架構* 在 AI 平台 上訓練及調整模型的程序完全相同！只要替換模型程式碼即可。

7. 清除所用資源

如要繼續使用這部筆電，建議在不使用時關機。在 Cloud 控制台的 Notebooks 使用者介面中，選取筆記本，然後選取「停止」：



如要刪除在本實驗室中建立的所有資源，請直接刪除筆記本執行個體，而不是停止執行個體。

使用 Cloud 控制台的導覽選單瀏覽至「儲存空間」，然後刪除您建立的兩個 bucket，以儲存模型資產。